

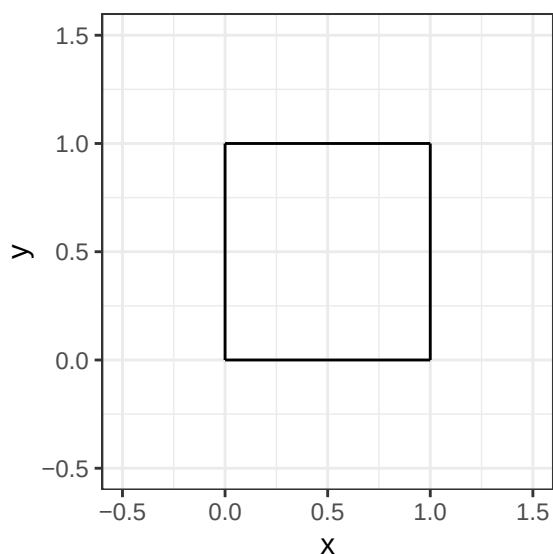
## Lab Three – Matrices and Data Frames in R

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with “**Solution:**”.
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.

Consider the unit square depicted in Figure 1.

```
1 p0 <- c(0, 0)
2 p1 <- c(1, 0)
3 p2 <- c(1, 1)
4 p3 <- c(0, 1)
5 ggplot() +
6   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +
7   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +
8   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +
9   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +
10  theme_bw() +
11  xlim(-0.5, 1.5) +
12  ylim(-0.5, 1.5) +
13  labs(x = "x", y = "y")
```

Figure 1: The unit square.



The unit square can be defined by two basis vectors

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

From these vectors, we can compute the corner vertices.

|                                         |                            |
|-----------------------------------------|----------------------------|
| $p_0 = (0, 0) = (x_0, y_0)$             | (The origin)               |
| $p_1 = v_1 = (x_1, y_1) = (1, 0)$       | (The first basis vector)   |
| $p_2 = v_1 + v_2 = (x_2, y_2) = (1, 1)$ | (The sum of basis vectors) |
| $p_3 = v_2 = (x_3, y_3) = (0, 1)$       | (The second basis vector)  |

Note that the segments that form the boundary go from  $p_0$  to  $p_1$  to  $p_2$  to  $p_3$  and back to  $p_0$ .

Consider the matrix  $M$  to be the matrix we get from binding the two basis vectors at the column,

$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

## 1 Question 1

Let's transform  $\mathbf{M}$  to demonstrate an interesting linear algebra property.

```
1 (M=matrix(data=c(1, 0, 0, 1), nrow=2, ncol=2, byrow=T)) # storing M to use for part b
```

```
      [,1] [,2]
[1,]     1     0
[2,]     0     1
```

### 1.1 Part a

Create the matrix  $\mathbf{A}$  in R.

$$\mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

**Solution**

```
1 (A = matrix(data=c(1, -7, 5, 9), nrow=2, ncol=2, byrow=T))
```

```
      [,1] [,2]
[1,]     1    -7
[2,]     5     9
```

### 1.2 Part b

Compute the product below in R and store it in an object  $\mathbf{T}$ .

$$\mathbf{T} = \mathbf{A}\mathbf{M}.$$

**Solution**

```
1 (T = A%%M)
```

```
      [,1] [,2]
[1,]     1    -7
[2,]     5     9
```

### 1.3 Part c

Create `basis.vector.1` (first column of  $\mathbf{T}$ ) and `basis.vector.2` (second column of  $\mathbf{T}$ ) in R.

**Solution**

```
1 (basis.vector1 = T[, 1]) # Select columns of T
```

```
[1] 1 5
```

```
1 (basis.vector2 = T[, 2])
```

```
[1] -7 9
```

## 1.4 Part d

Compute the points  $p_0$ ,  $p_1$ ,  $p_2$ , and  $p_3$ .

**Solution**

```
1 (p0 = c(0, 0))  
  
[1] 0 0  
1 (p1 = c(1, 5))  
  
[1] 1 5  
1 (p2 = c(-6, 14))  
  
[1] -6 14  
1 (p3 = c(-7, 9)) # Store points as vectors  
  
[1] -7 9
```

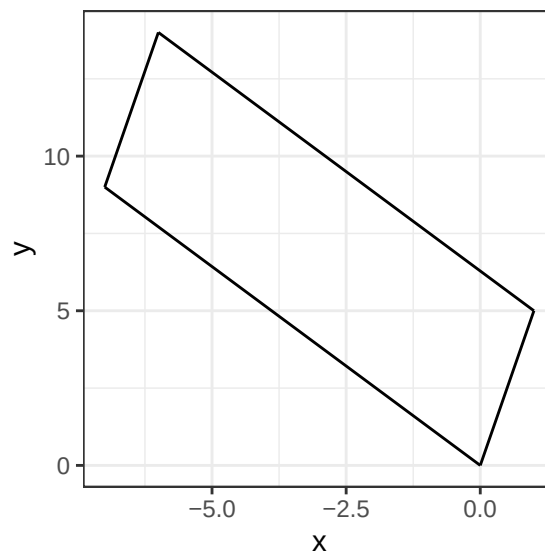
## 1.5 Part e

Copy and paste the code for the unit square. Edit the code to draw the segments that form the boundary based on the points in Part d.

**Solution**

```
1 ggplot() +  
2   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +  
3   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +  
4   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +  
5   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +  
6   theme_bw() +  
7   # xlim(-15, 15) +  
8   # ylim(-5, 25) +  
9   labs(x = "x", y = "y") #Plot unit square based on new points in part d
```

Figure 2: Shape 2.



## 1.6 Part f

Interestingly, the determinant of  $\mathbf{A}$  gives us the area of the shape plotted in Part e. Find the determinant of  $\mathbf{A}$ .

**Solution**

```
1 det(A)
```

```
[1] 44
```

## 2 Question 2

Recomplete Question 1 using a new matrix for **A**. This matrix is not as well behaved.

### 2.1 Part a

Create the matrix A in R.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

**Solution**

```
1 (A = matrix(data = c(1, 2, 2, 4), nrow=2, ncol=2, byrow=T))
```

```
      [,1] [,2]  
[1,]     1     2  
[2,]     2     4
```

### 2.2 Part b

Compute the product below in R and store it in an object T.

$$\mathbf{T} = \mathbf{A}\mathbf{M}.$$

**Solution**

```
1 (T = A%%M)
```

```
      [,1] [,2]  
[1,]     1     2  
[2,]     2     4
```

### 2.3 Part c

Create **basis.vector.1** (first column of T) and **basis.vector.2** (second column of T) in R.

**Solution**

```
1 (basis.vector1 = T[, 1])
```

```
[1] 1 2
```

```
1 (basis.vector2 = T[, 2]) # Select columns of T
```

```
[1] 2 4
```

### 2.4 Part d

Compute the points  $p_0$ ,  $p_1$ ,  $p_2$ , and  $p_3$ .

**Solution**

```
1 (p0 = c(0, 0))
```

```
[1] 0 0
```

```
1 (p1 = c(1, 2))
```

```
[1] 1 2
```

```

1 (p2 = c(3, 6))

[1] 3 6
1 (p3 = c(2, 4)) # Store points as vectors

[1] 2 4

```

## 2.5 Part e

Copy and paste the code for plotting the unit square. Edit the code to draw the segments that form the boundary based on the points in Part d.

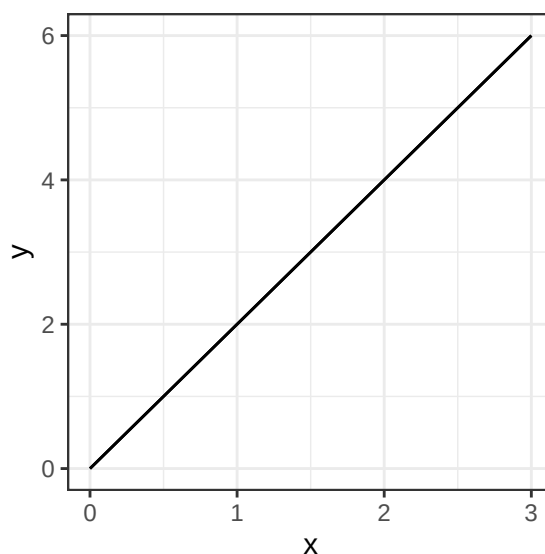
### Solution

```

1 ggplot() +
2   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +
3   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +
4   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +
5   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +
6   theme_bw() +
7   # xlim(-15, 15) +
8   # ylim(-5, 25) +
9   labs(x = "x", y = "y") # Plot unit square with points from part d

```

Figure 3: Shape 3.



## 2.6 Part f

Interestingly, the determinant of  $\mathbf{A}$  gives us the area of the shape plotted in Part e. Find the determinant of  $\mathbf{A}$ .

### Solution

```

1 det(A)

[1] 0

```

The area is 0 because the shape is a straight line.

## 3 Question 3

We can conduct a shear transformation that keeps the area of the shape the same but horizontally or vertically slants the object.

In fact, this is one of the ways artists can make something look like it's leaning or being pushed. Other applications where this is useful include fluid dynamics and crystallography, where the area of a deformed object must remain constant, or in photography where perspective in photos can be adjusted.

A shear transformation is implemented by altering the basis vectors we start with. For vertical slanting,

$$\mathbf{M}_v = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

and for horizontal slanting

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}.$$

In both cases,  $k \in \mathbb{R}$  is called a shear factor and its sign determines the direction of the slant and its magnitude determines the severity of the slant.

### 3.1 Part a

Use the following to generate a new shape. Compare the shape and its area to your answer in Question 1.

$$\mathbf{M}_v = \begin{bmatrix} 1 & k = 0.5 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

#### Solution

```
1 M = matrix(data = c(1, 0.5, 0, 1), nrow=2, ncol=2, byrow=T)
2 A = matrix(data = c(1, -7, 5, 9), nrow=2, ncol=2, byrow=T)
3 (T = A%*%M) #Matrix multiplication
```

```
      [,1] [,2]
[1,]      1 -6.5
[2,]      5 11.5
```

```
1 det(T) # Calculate area
```

```
[1] 44
```

```
1 (p0 = c(0, 0))
```

```
[1] 0 0
```

```
1 (p1 = c(1, 5))
```

```
[1] 1 5
```

```
1 (p2 = c(-5.5, 16.5))
```

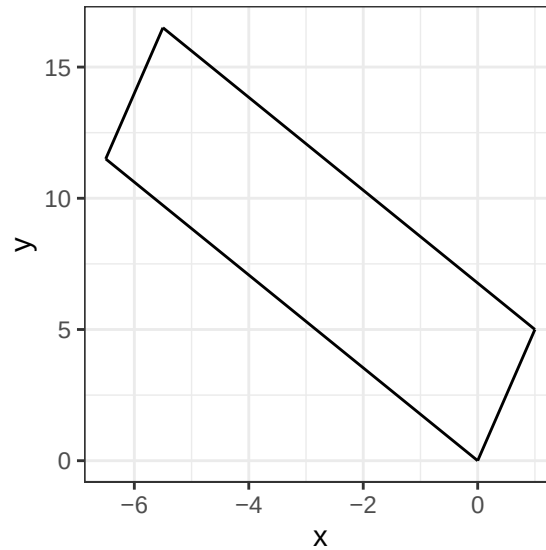
```
[1] -5.5 16.5
```

```
1 (p3 = c(-6.5, 11.5)) # Store points as vectors
```

```
[1] -6.5 11.5
```

```
1 ggplot() +
2   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +
3   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +
4   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +
5   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +
6   theme_bw() +
7   # xlim(-15, 15) +
8   # ylim(-5, 25) +
9   labs(x = "x", y = "y") # Plot unit square after vertical shearing
```

Figure 4: Shape 4.



The area of this shape is the same as the area of the shape in question 1, although it looks different. Through a vertical shear, the longer sides of this shape have been stretched, while the shorter sides have become shorter, distorting the parallelogram a bit.

### 3.2 Part b

Use the following to generate a new shape. Compare the shape and its area to your answer in Questions 1 and the vertical shear transform in part (a).

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k=0.5 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

#### Solution

```
1 M = matrix(data = c(1, 0, 0.5, 1), nrow=2, ncol=2, byrow=T)
2 (T = A%%M) # Matrix multiplication
```

```
      [,1] [,2]
[1,] -2.5  -7
[2,]  9.5   9
```

```
1 det(T) # Calculate area
```

```
[1] 44
```

```
1 (p0 = c(0, 0))
```

```
[1] 0 0
```

```
1 (p1 = c(-2.5, 9.5))
```

```
[1] -2.5  9.5
```

```
1 (p2 = c(-9.5, 18.5))
```

```
[1] -9.5 18.5
```

```
1 (p3 = c(-7, 9)) # Store the points as vectors
```

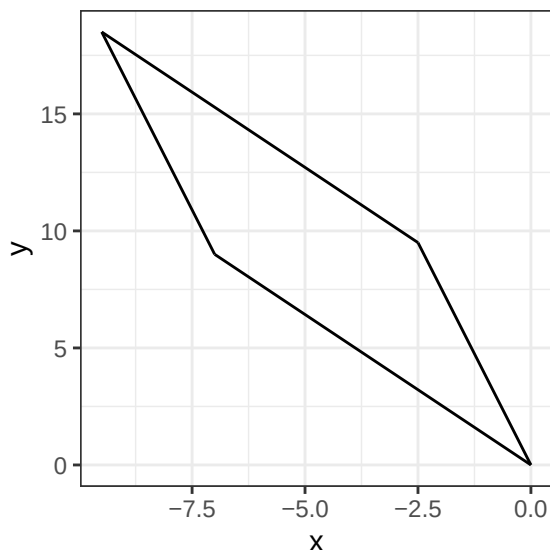
```
[1] -7  9
```

```

1 ggplot() +
2   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +
3   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +
4   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +
5   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +
6   theme_bw() +
7   # xlim(-15, 15) +
8   # ylim(-5, 25) +
9   labs(x = "x", y = "y") # Plot the unit square with a horizontal shear

```

Figure 5: Shape 5.



The area of this shape is the same as the area of the shape in question 1 and the vertical shear in part a, although it looks different. This time, through a horizontal shear, the shorter sides of this shape have been stretched, while the longer sides have become shorter, stretching the parallelogram in the opposite way.

### 3.3 Part c

Can we shear vertically and horizontally at the same time? Use the following to generate a new shape. Compare the shape and its area to your answers in parts (a) and (b).

$$\mathbf{M}_{vh} = \begin{bmatrix} 1 & k = 0.5 \\ k = 0.5 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

#### Solution

```

1 M = matrix(data = c(1, 0.5, 0.5, 1), nrow=2, ncol=2, byrow=T)
2 (T = A%*%M) # Matrix Multiplication

```

```

      [,1] [,2]
[1,] -2.5 -6.5
[2,]  9.5 11.5

```

```

1 det(T) # Calculate area

```

```

[1] 33

```

Area does not equal 44 this time (has changed!)

```

1 (p0 = c(0, 0))

```

```

[1] 0 0

```



```

1 (p1 = c(-2.5, 9.5))

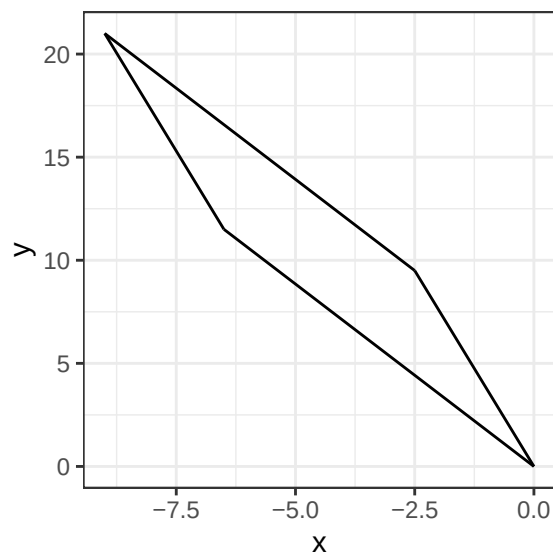
[1] -2.5  9.5
1 (p2 = c(-9, 21))

[1] -9 21
1 (p3 = c(-6.5, 11.5)) # Store the points as vectors

[1] -6.5 11.5
1 ggplot() +
2   geom_segment(aes(x = p0[1], y = p0[2], xend = p1[1], yend = p1[2])) +
3   geom_segment(aes(x = p1[1], y = p1[2], xend = p2[1], yend = p2[2])) +
4   geom_segment(aes(x = p2[1], y = p2[2], xend = p3[1], yend = p3[2])) +
5   geom_segment(aes(x = p3[1], y = p3[2], xend = p0[1], yend = p0[2])) +
6   theme_bw() +
7   # xlim(-15, 15) +
8   # ylim(-5, 25) +
9   labs(x = "x", y = "y") # Plot the unit square with both a horizontal and vertical shear

```

Figure 6: Shape 6.



This time both the area of the shape and its appearance have changed. Since we applied both a vertical and horizontal shear, both the short and long sides have been shortened. The shape therefore looks very compressed, and the area changed from 44 (in question 1 and parts a and b) to 33.

## 4 Question 4

### 4.1 Part a

Create a data frame `ladder` with the following data. Note your data frame should have three columns; I split it to save vertical space here.

| x | y  | group | x | y  | group |
|---|----|-------|---|----|-------|
| 0 | 0  | 1     | 1 | 8  | 6     |
| 0 | 20 | 1     | 0 | 10 | 7     |
| 1 | 0  | 2     | 1 | 10 | 7     |
| 1 | 20 | 2     | 0 | 12 | 8     |
| 0 | 2  | 3     | 1 | 12 | 8     |

| x | y | group | x | y  | group |
|---|---|-------|---|----|-------|
| 1 | 2 | 3     | 0 | 14 | 9     |
| 0 | 4 | 4     | 1 | 14 | 9     |
| 1 | 4 | 4     | 0 | 16 | 10    |
| 0 | 6 | 5     | 1 | 16 | 10    |
| 1 | 6 | 5     | 0 | 18 | 11    |
| 0 | 8 | 6     | 1 | 18 | 11    |

### Solution

```

1 x = c(0,0,1,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1)
2 y = c(0,20,0,20,2,2,4,4,6,6,8,8,10,10,12,12,14,14,16,16,18,18)
3 group = rep(1:11, each=2) # Store the data in vectors
4 ladder = data.frame(x, y, group) # Create a data frame

```

## 4.2 Part b

Remove `#|eval: false` in the code below. The result should look like a ladder and a very rectangular house if part a is correct.

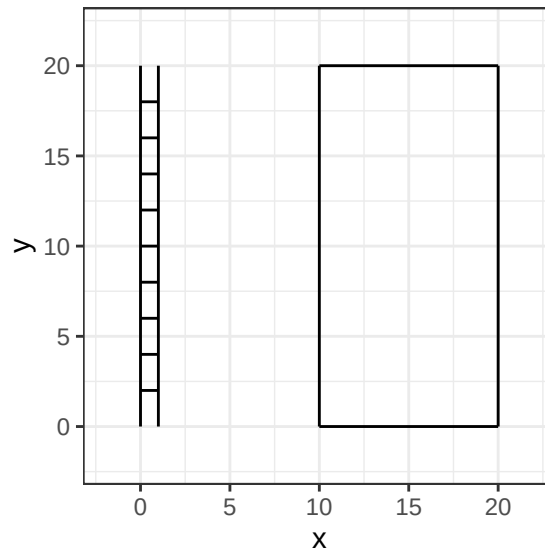
### Solution

```

1 ggplot() +
2   geom_path(data = ladder,
3             aes(x = x, y = y, group = group)) +
4     geom_segment(aes(x = 10, xend = 10,
5                      y=0, yend = 20))+
6   geom_segment(aes(x = 10, xend = 20,
7                      y=20, yend = 20))+
8   geom_segment(aes(x = 20, xend = 20,
9                      y=0, yend = 20))+
10  geom_segment(aes(x = 10, xend = 20,
11                  y=0, yend = 0)) +
12  theme_bw() +
13  xlim(-2,22) +
14  ylim(-2,22) # Plot the ladder

```

Figure 7: A ladder.



### 4.3 Part c

Use `as.matrix()` to create a  $22 \times 2$  matrix **A** containing the **x** and **y** observations from the `ladder` data frame. Use matrix multiplication to compute

$$\mathbf{T} = \mathbf{A}\mathbf{M}_h.$$

Recall,

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$$

and use  $k = 0.5$ .

#### Solution

```
1 A = as.matrix(ladder[, c("x", "y")]) # Make matrix using column one and two of the data frame
2 M = matrix(data = c(1, 0, 0.5, 1), nrow=2, ncol=2, byrow=T)
3 (T = A%*%M)
```

```
      [,1] [,2]
[1,]     0     0
[2,]    10    20
[3,]     1     0
[4,]    11    20
[5,]     1     2
[6,]     2     2
[7,]     2     4
[8,]     3     4
[9,]     3     6
[10,]    4     6
[11,]    4     8
[12,]    5     8
[13,]    5    10
[14,]    6    10
[15,]    6    12
[16,]    7    12
[17,]    7    14
[18,]    8    14
[19,]    8    16
[20,]    9    16
[21,]    9    18
[22,]   10    18
```

### 4.4 Part d

Create a data frame `leaning.ladder` with the **x** and **y** equal to the first and second column of **T**, respectively, and the same values of **group** from `ladder`.

#### Solution

```
1 x = T[, 1]
2 y = T[, 2] # Select columns of T
3 leaning.ladder = data.frame(x, y, group) # Data frame with new values of x and y
```

## Part e Remove `eval: false` in the code below. The result should look like a ladder and a very rectangular house if part a is correct. The new leaning ladder should look like it is leaning on the house.

#### Solution

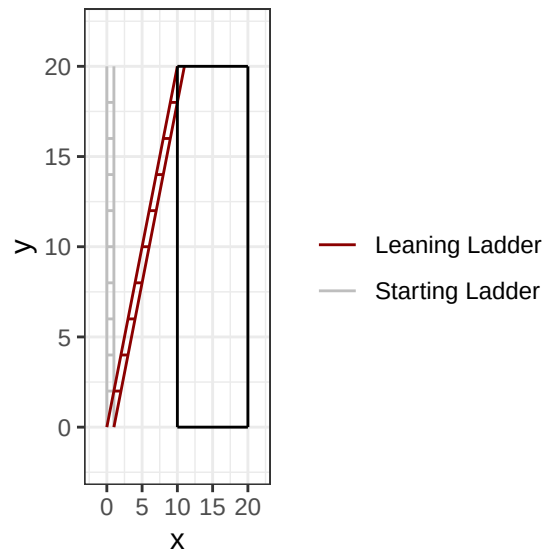
```
1 ggplot() +
2   geom_path(data = ladder,
3             aes(x = x, y = y, group = group, color = "Starting Ladder")) +
4   geom_path(data = leaning.ladder,
5             aes(x = x, y = y, group = group, color = "Leaning Ladder")) +
6   geom_segment(aes(x = 10, xend = 10,
7                    y=0, yend = 20))+
```

```

8   geom_segment(aes(x = 10, xend = 20,
9                     y=20, yend = 20))+
10  geom_segment(aes(x = 20, xend = 20,
11                    y=0, yend = 20))+
12  geom_segment(aes(x = 10, xend = 20,
13                    y=0, yend = 0)) +
14  theme_bw() +
15  xlim(-2,22) +
16  ylim(-2,22) +
17  scale_color_manual("", values=c("darkred", "grey")) # Plot leaning ladder

```

Figure 8: A ladder and a leaning ladder.



## 4.5 Discussion

From questions 1 to 4 we see the capabilities of R in creating and altering images using matrices, data frames, and more. Starting from doing basic matrix formation, multiplication, and determinants we moved to shearing in question 3. Here, we see how changing the identity matrix,  $M$ , slightly can stretch resulting shapes through matrix multiplication (whether that is vertically, horizontally, or both). Lastly, in question 4, a data frame was used to create the ladder in figure 7 from the table values given in part a. Interestingly, by then storing the columns of  $x$  and  $y$  from the table as columns in a 22 by 2 matrix, we could then use shearing again to alter the original ladder to slant and look as if it is leaning without changing its area (figure 8).

## 4.6 Citations

Only used R Core Team (2025) and related packages for this assignment.

## References

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org/>.